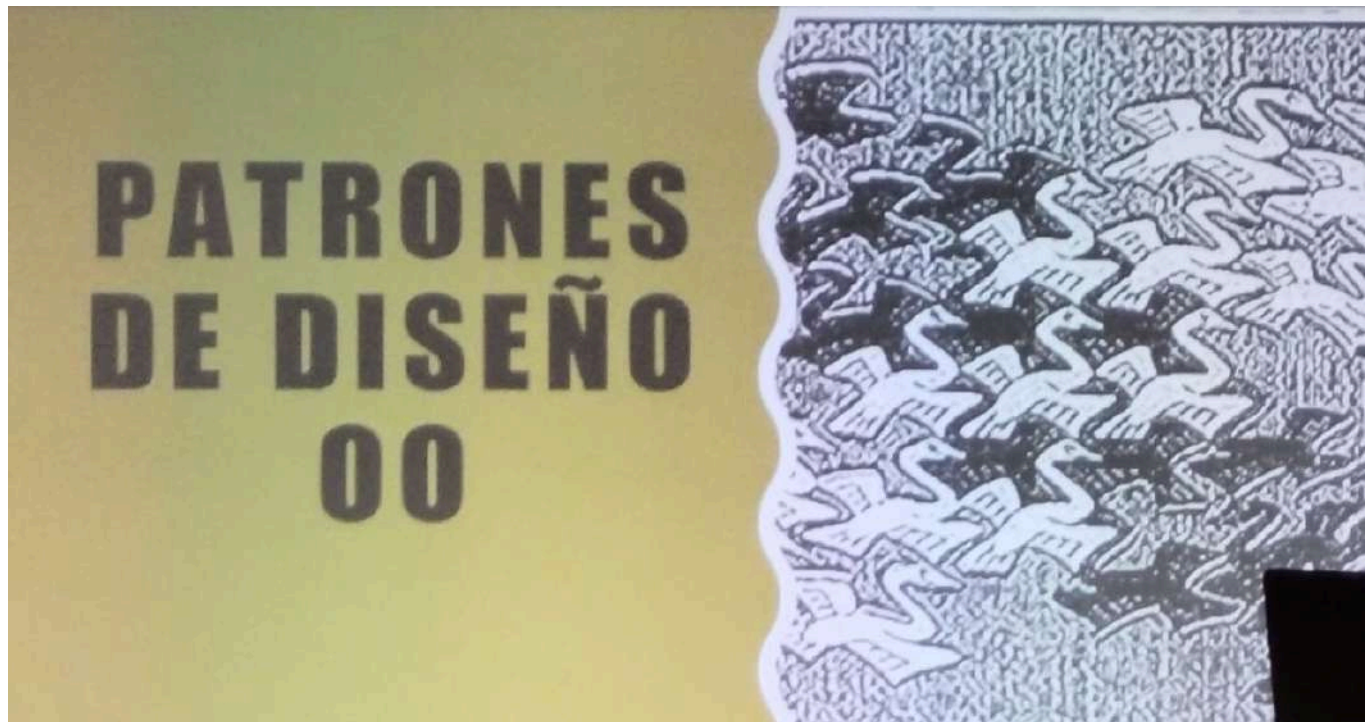


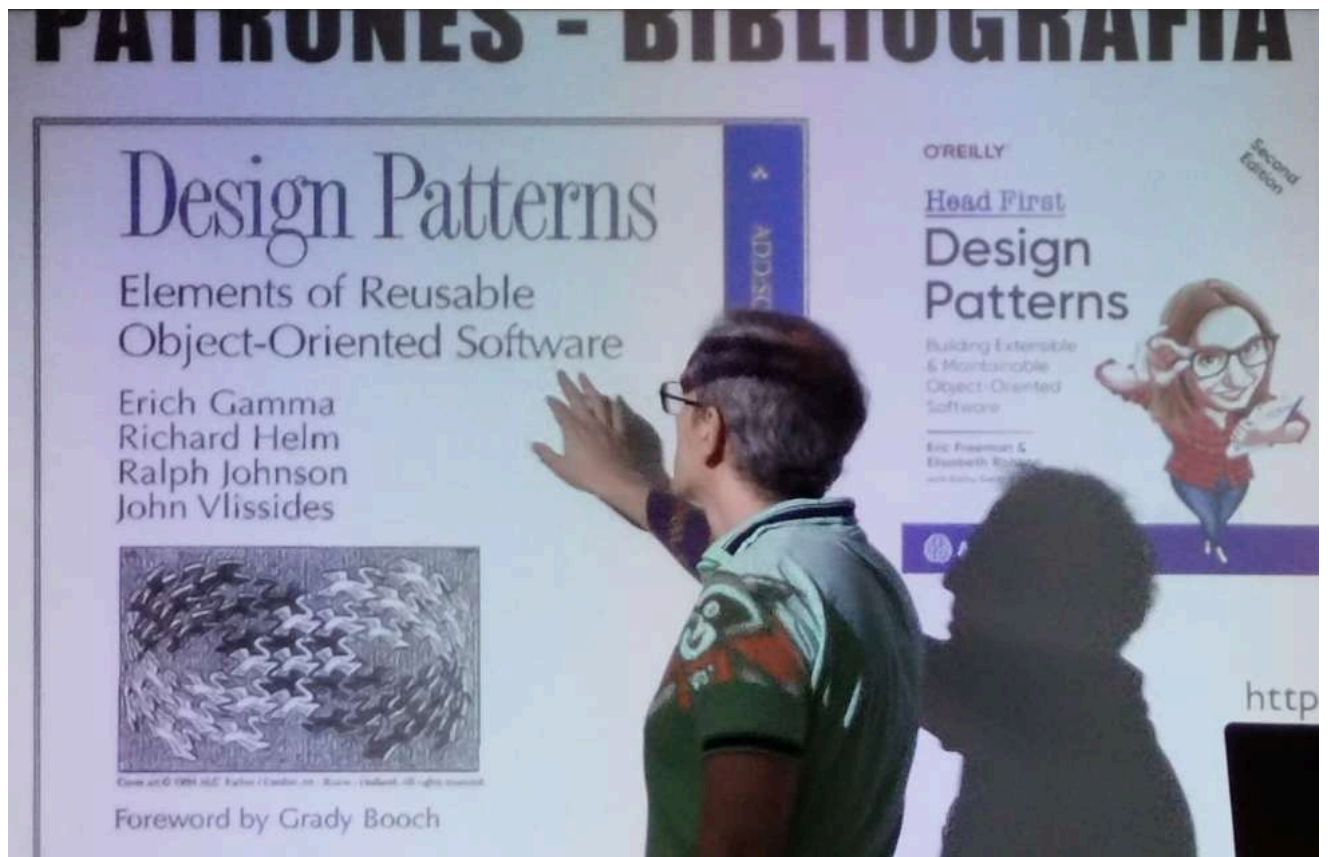
CLASE - 22/10

PATRONES DE DISEÑO OO

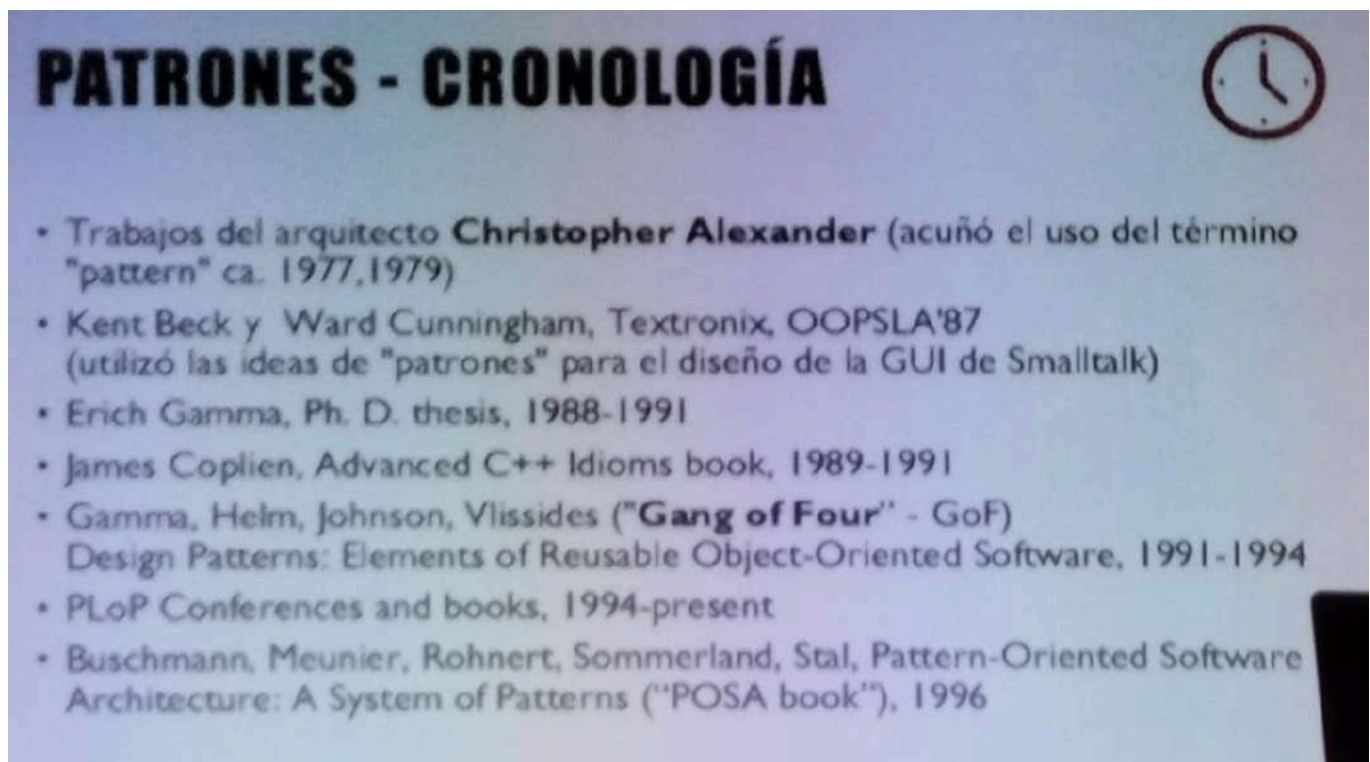


(la cabra juani se jugó las anotaciones)

PATRONES



CRONOLOGÍA - PATRONES



(había un par de fotos aca)

DEFINICIONES - PATRONES

DEFINICIONES

- ...es un **modelo** que sirve de muestra para sacar otra cosa igual [RAE]
- ... describe un **problema** que ocurre una y otra vez en nuestro entorno, y describe el **núcleo de la solución** a dicho problema, de modo de que pueda utilizarse dicha solución millones de veces, sin tener que realizar lo mismo dos veces [Alexander]
- ... es una **descripción de objetos** que se comunican y clases a la medida para **resolver un problema de diseño general** en un **contexto particular** [Gamma et al]



-Es un modelo que sirve de muestra para otra cosa igual

-Un patrón describe un problema que ocurre una y otra vez en nuestro entorno. La idea con el es usarlo como herramienta, no que nuestra solución sea exactamente igual

-No se da una receta si no se describen las características básicas para abordar el problema. Re Usarlas cuando las necesitemos.

-En síntesis, el patrón es un par problema-solución. Para cada problema vamos a aplicar una solución base para muchos problemas.

-ofrecemos soluciones bases masomenos parecidas a la solución rigina, aunque no siempre va a salir idéntica

-Cada solución es para un determinado contexto (entorno), no son aplicables en todos ellos, si no en uno particular

-vamos a manejar de la más genérica a la más específica en nuestro campo

PROPIEDADES - PATRONES

PATRONES - PROPIEDADES	
 Si	 No
• Proveen un vocabulario común • Proveen un modo abreviado para comunicar efectivamente principios complejos • Ayudan a documentar la arquitectura del software • Capturan la parte esencial de un diseño en forma compacta • Muestran más de una solución posible • Describen abstracciones de software	• Proveen una solución exacta • Solucionan todos los problemas de diseño • Solo se aplican a diseños orientados a objetos

Verdaderas (representativas):

- Vocabulario común
- Comunicación efectiva
- Documentar arquitectura de SW: una arquitectura puede estar formada por varios patrones
- Compactan el diseño: nos permiten enfocarnos en partes del sistema y como vamos a ir resolviendo cada una.
- Más de una solución posible
- Abstracciones de SW

Falsas (no representativas):

- Solución exacta
- Solución para todos los problemas de diseño
- Solo se aplican a diseño orientados a objetos

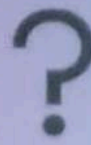
COMPONENTES - PATRONES

PATRONES - COMPONENTES

Según Alexander, todo patrón tiene **3 elementos esenciales**:



Contexto



Problema



Solución

-Menos importante: contexto (como la solución no es universal, se aclara el contexto en el que se aplica)

-Los elementos que deben estar a la hora de definir un patrón son: contexto - problema - solución

TIPOS DE PATRONES DE SOFTWARE

TIPOS DE PATRONES DE SOFTWARE

- de diseño (diseño de software)
 - Arquitectónicos (estructura de sistemas) [Buschmann-POSA]
 - Diseño (microarquitecturas) [Gamma-GoF]
 - Codificación o Idiomas o Modismos (Idioms) (bajo nivel)
- de análisis (modelos de análisis recurrentes y reutilizables) [Fowler]
- de organización (estructura de organizaciones/proyectos)
- de proceso (diseño de procesos de software)
- específicos del dominio

De diseño

- Arquitectónicos:: los que ya vimos (cliente-servidor, microservicios, etc)
- Diseño
- Codificación (más bajo nivel): También se conocen como idiomas/modismos

De análisis

De organización

FORMA DE ALEXANDER CANÓNICA

FORMA DE ALEXANDER (CANÓNICA)	
Nombre	nombre significativo
Problema	la descripción del problema
Contexto	una situación dada en la que aparece el problema
Fuerzas	una descripción de las fuerzas relevantes y las restricciones que se aplican
Solución	una solución probada al problema
Ejemplos	ejemplo de aplicación del patrón
Contexto resultante	estado del sistema luego de la aplicación del patrón
Racionalidad	explicación de los pasos o reglas del patrón
Patrones relacionados	relaciones estáticas y dinámicas con otros patrones
Usos conocidos	ocurrencias del patrón y su aplicación en sistemas existentes

-Están los componentes básicos del patrón más el nombre (que sirve como forma de comunicación), fuerzas (características que van a intervenir en el contexto para ayudar a caracterizar tanto el problema como la solución posible)

-Otra cosa importante son los ejemplos. Ayuda a contextualizar. Aparte de citar el ejemplo, es necesario explicarlo, en ocasiones.

-Además pueden existir racionalidad, patrones relacionados (patrones que se usan en conjunto con otros).

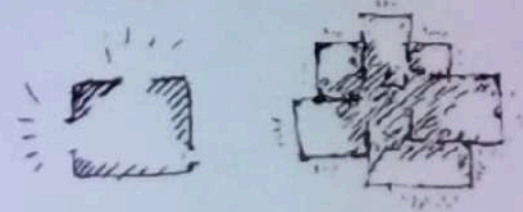
Algunos patrones funcionan mejor con otros.

EJEMPLO FORMA DE ALEXANDER

EJEMPLO FORMA DE ALEXANDER

- **Nombre:** *Light on Two Sides of Every Room (159)*
- **Contexto:** Habitación en un edificio con iluminación natural.
- **Problema:** ¿Cómo lograr que una habitación se sienta agradable ...?
- **Fuerzas:**
 - La luz natural es fundamental para el confort y la percepción del espacio.
 - Una sola fuente de luz lateral produce sombras fuertes y zonas oscuras.
 - Dos fuentes de luz natural desde lados opuestos crean una iluminación suave y cambiante a lo largo del día.
 - Pero agregar ventanas en más de una pared puede ser difícil por el diseño estructural o la orientación del edificio.
- **Solución:**
 - Diseña las habitaciones principales de modo que tengan ventanas en al menos dos lados distintos.

each room has light on two sides



FORMATO GOF

FORMATO GOF

Nombre y Clasificación	Nombre y categoría en la que encuadra el patrón.
Intención	Qué hace el patrón y cuando funciona la solución.
También conocido como	Otros nombres con que el patrón es conocido.
Motivación	El problema de diseño y como la estructura de clases y objetos del patrón soluciona el problema.
Aplicabilidad	Situaciones donde el patrón puede ser aplicado.
Estructura	Representación gráfica de las clases en el patrón.
Participantes	Las clases y objetos participantes y sus responsabilidades.
Colaboraciones	De los participantes para llevar adelante sus responsabilidades.
Consecuencias	Compromisos y consecuencias de la aplicación del patrón.
Implementación	Heurísticas y técnicas para implementar el patrón.
Código de ejemplo	Fragmentos de código en algún lenguaje (Smalltalk, C++, Java) mostrando una posible implementación.
Usos conocidos	Ejemplos de utilización del patrón en sistemas reales.
Patrones relacionados	Otros patrones que se relacionan dinámica y estáticamente.

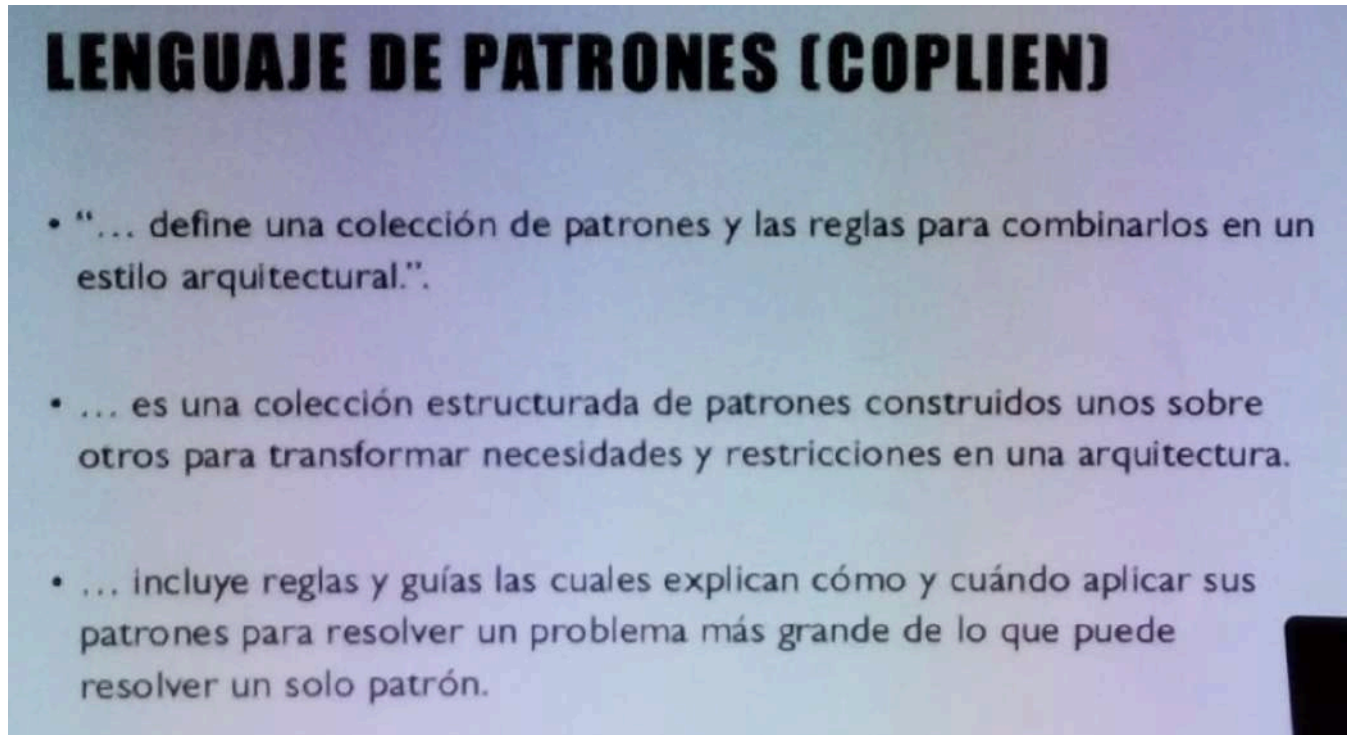
-Clasificación: clasifica los patrones en 3 categorías diferentes.

-Intención: Que es lo que hace el patrón, que es lo que pretende resolver

-Aplicabilidad: sigue hablando de contexto, pero cambia el nombre

- Estructura: representación gráfica que va a tener la solución, en general, la forma será un diagrama de clases
- Debemos tener todo lo que aporte al entendimiento de la solución (consecuencias, implementaciones, código de ejemplo, etc)

LENGUAJE DE PATRONES (COPLIEN)



- Se presenta esta definición porque hay definiciones de otros autores que son similares

CATÁLOGO Y SISTEMAS DE PATRONES

CATÁLOGO Y SISTEMAS DE PATRONES

- [Buschmann, POSA]
- Catálogo de patrones: ... colección de patrones relacionados, donde los patrones se subdividen en un número de categorías ...
- Sistema de patrones: ... conjunto cohesivo de patrones relacionados, los que trabajan juntos para soportar la construcción y evolución de una arquitectura completa...

-Catálogo de patrones habla de una conexión de patrones relacionados. Existe una relación, pero la misma no es fuerte (Gasmann)

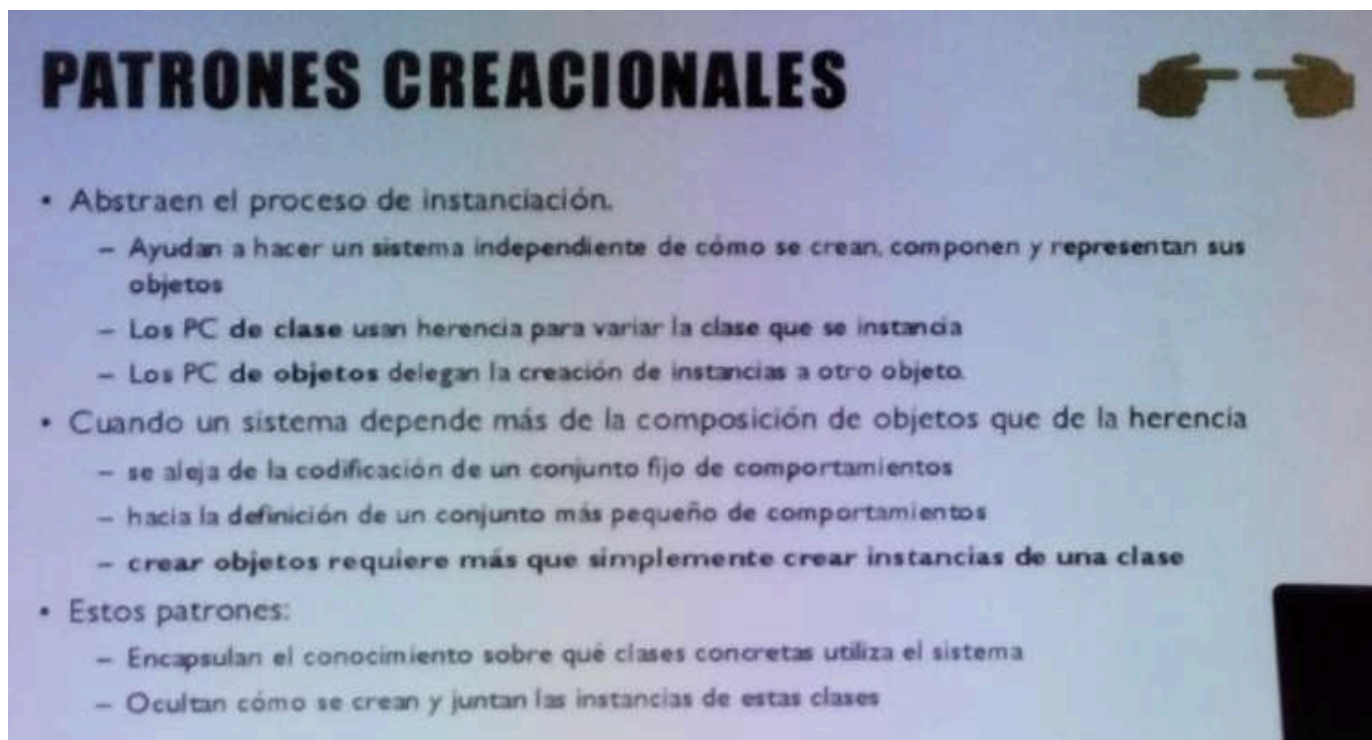
-Sistema de patrones es un conjunto de patrones más cohesivo, osea los patrones se usan en conjunto para elaborar la solución

PATRONES GOF (cuadro)

		Propósito		
		Creadacional	Estructural	Comportamiento
Alcance	Clase	Factory Method	Adapter	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

- Es un conjunto de 23 patrones. Es importante saber explicar y aplicar los patrones que se ven en las clases. Van a aparecer más patrones durante el estudio de ellos
- La clasificación más utilizada para los patrones será la de propósito.
- Patrones creacionales, patrones estructurales y patrones de comportamiento (clasificación según propósito)
- Patrones basados en clases, patrones basados en objetos (clasificación según alcance)--> |creacionales
- Los patrones basados en clase basan su solución en una herencia mientras que los que están basados en objetos basan su solución en agregación y composición
- Adapter estaba en dos clasificaciones distintas, por lo cual es importante su estudio

PATRONES CREACIONALES



PATRONES CREACIONALES

- Abstraen el proceso de instanciación.
 - Ayudan a hacer un sistema independiente de cómo se crean, componen y representan sus objetos
 - Los PC de **clase** usan herencia para variar la clase que se instancia
 - Los PC de **objetos** delegan la creación de instancias a otro objeto.
- Cuando un sistema depende más de la composición de objetos que de la herencia
 - se aleja de la codificación de un conjunto fijo de comportamientos
 - hacia la definición de un conjunto más pequeño de comportamientos
 - **crear objetos requiere más que simplemente crear instancias de una clase**
- Estos patrones:
 - Encapsulan el conocimiento sobre qué clases concretas utiliza el sistema
 - Ocultan cómo se crean y juntan las instancias de estas clases

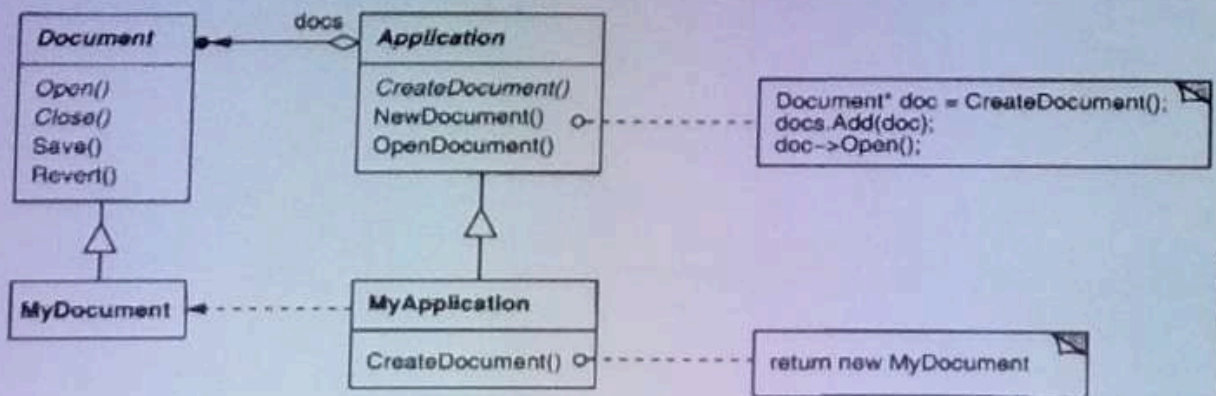
- La instanciación no es tan trivial.
- La categorización de patrones es una forma de organizar el estudio, y los creacionales ayudarán a adecuar cada uno al contexto propicio

FACTORY METHOD

FACTORY METHOD



- **Intención:** definir una interfaz para crear un objeto, pero dejar que las subclasses decidan qué clase instanciar. Factory Method permite que una clase difiera la creación de instancias a subclasses.



-Aclaración: No se puede afirmar a ciencia cierta qué patrones se van a usar.

-Los Frameworks se basan en clases abstractas que dejarán que las subclasses decidan a qué elementos van a instanciar.

(otro power)

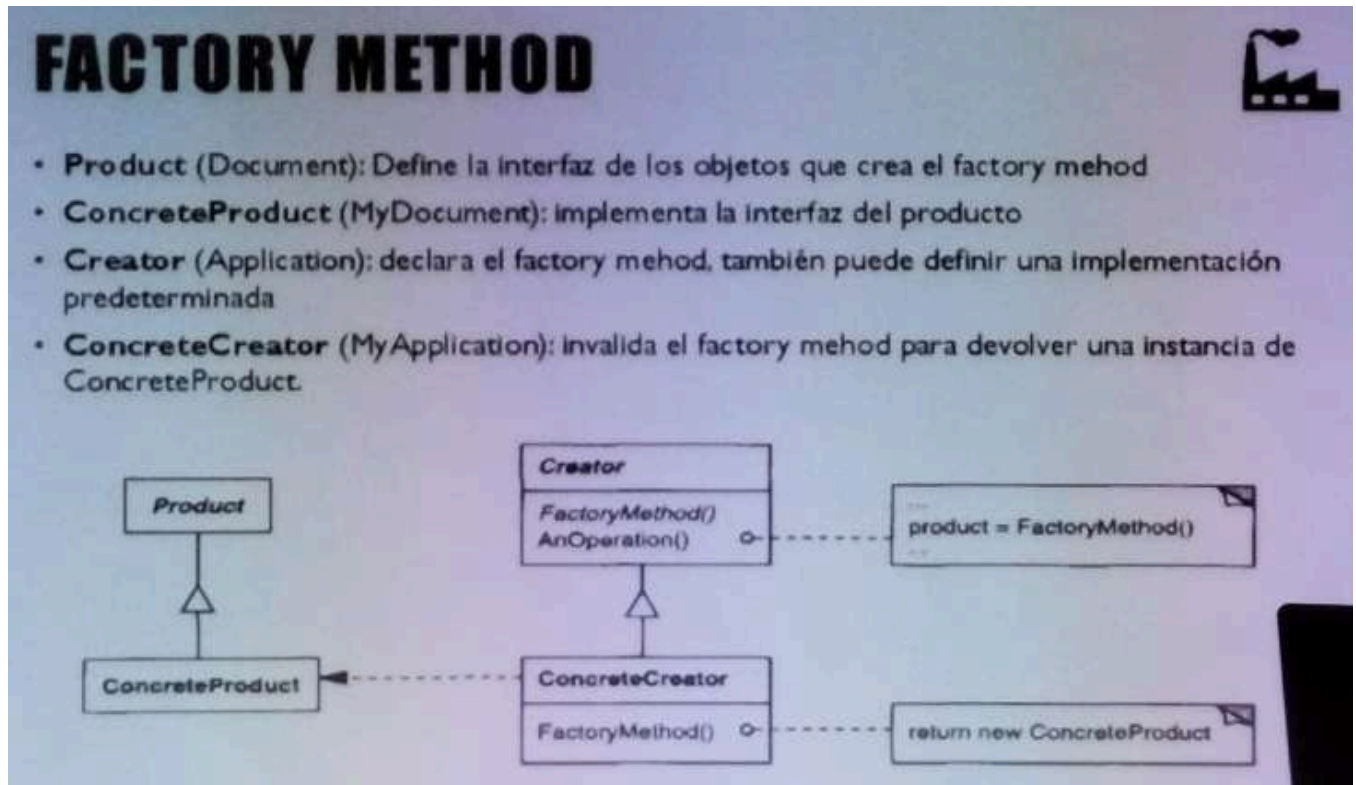
FACTORY METHOD



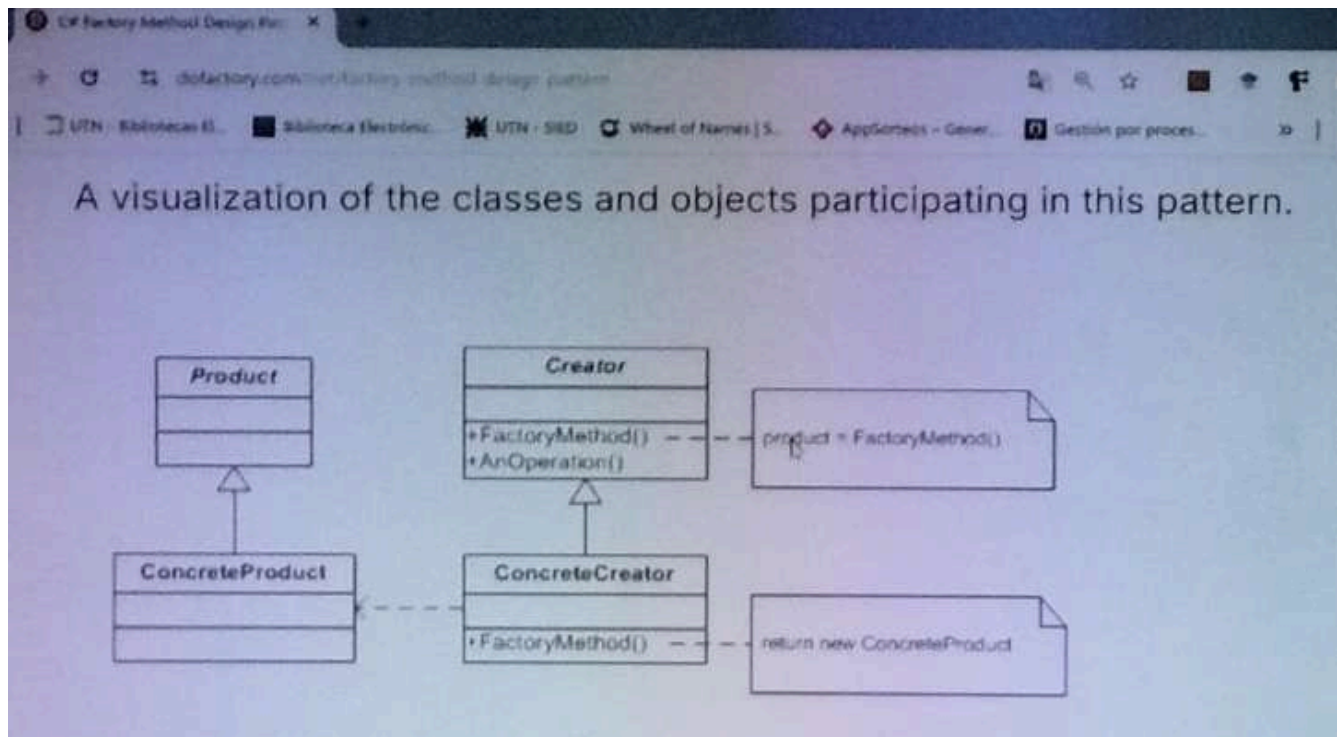
• Aplicabilidad:

- Una clase **no puede anticipar la clase de objetos que debe crear.**
- Una clase desea que sus subclasses especifiquen los objetos que crea.
- Las clases delegan la responsabilidad a una de varias subclasses auxiliares y desea localizar el conocimiento de qué subclase auxiliar es el delegado.

(otro power)



- En la forma/estructura básica de un patrón existen creadores y creadores concretos.
- Factory method es un método que existe en una clase abstracta que luego se implementa con overwriting (override)
- estos son los 4 participantes que propone



- **Product (Page)**
 - defines the interface of objects the factory method creates
- **ConcreteProduct (SkillsPage, EducationPage, ExperiencePage)**
 - implements the Product interface
- **Creator (Document)**
 - declares the factory method, which returns an object of type Product. Creator may also define a default implementation of the factory method that returns a default ConcreteProduct object.
 - may call the factory method to create a Product object.
- **ConcreteCreator (Report, Resume)**

<https://www.dofactory.com/net/design-patterns>

(Mostró un ejemplo en código de esto, mostrando cómo se implementan estas clases abstractas con clases concretas. Era un ejemplo sobre Product, Creator, etc)

Se aplica el principio de Sustitución de Liskov y Polimorfismo a lo largo del código

El creador genérico no puede saber que producto crear porque depende del concreto para poder efectuar ello.

Los Templates no vamos a dar en la materia, pero los generics en C# vendrían a ser los templates. En UML los templates se ven más complicado de lo que en realidad son

Los CreadoresConcretos son los que saben que productos crear

SINGLETON

SINGLETON

1

- **Analizar patrón y encontrar un posible ejemplo**

- única instancia
- acceso global
- método de clase o estático para acceder
- se utiliza un constructor privado, el cual normalmente es público. Pregunta si la instancia había sido creada anteriormente, sino se crea la crea

PATRONES ESTRUCTURALES

PATRONES ESTRUCTURALES



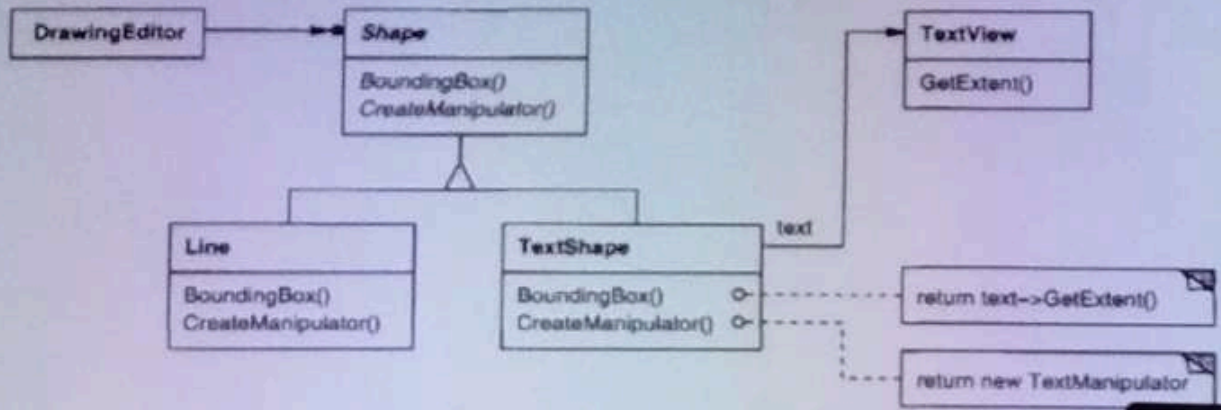
- Se refieren a cómo se componen las clases y los objetos para formar estructuras más grandes.
- Los de clase utilizan la herencia para componer interfaces o implementaciones.
- Los de objeto describen formas de componer objetos para obtener nuevas funcionalidades.
- La flexibilidad añadida de la composición de objetos proporciona la capacidad de cambiar la composición en tiempo de ejecución.

- me permiten crear jerarquía más grandes
- este tipo de jerarquías se pueden crear mediante herencia, agregación o composición

ADAPTER

ADAPTER

- **Intención:** Convertir la interfaz de una clase en otra interfaz que los clientes esperan. El adaptador permite que las clases trabajen juntas, ya que de otro modo no podrían debido a interfaces incompatibles.



-es la interfaz como tal, propuesta por la clase. No necesariamente a través de un clasificador Interface. (en este libro se encuentra Interfaz como aquello que la clase expone hacia afuera)

-pueden existir clases que podemos reutilizar pero no son compatibles con las clases existentes

EJEMPLO

-este Editor tiene una clase padre

-si yo creo un elemento de este tipo por LSP y Polimorfismo va a funcionar

Aplicabilidad(power)

ADAPTER

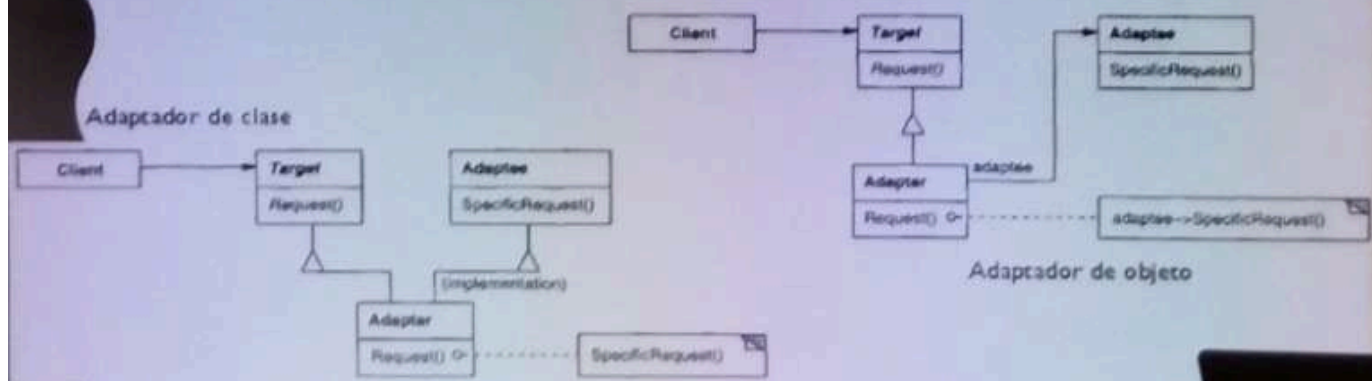
• Aplicabilidad

- **Desea utilizar una clase existente y su interfaz no coincide con la que necesita.**
- Desea crear una clase reutilizable que coopere con clases no relacionadas o imprevistas, es decir, clases que no necesariamente tienen interfaces compatibles.
- (solo adaptador de objetos) Debe usar varias subclases existentes, pero no es práctico adaptar su interfaz subclassificando a todos. Un adaptador de objetos puede adaptar la interfaz de su clase principal.

(power)

ADAPTER

- **Target (Shape):** define la interfaz específica del dominio que utiliza Client.
- **Client (DrawingEditor):** colabora con objetos conformes a la interfaz de Target.
- **Adaptee (TextView):** define una interfaz existente que necesita adaptación.
- **Adapter (TextShape):** adapta la interfaz de Adaptee a la interfaz de Target.



-venía en dos variantes, una era de clase y otra de objeto

-en ambas variantes tenemos 4 tipos

-el objetivo es aquello que lo que queremos adaptar, aca se denomina como Target

(el uso de Cliente en el libro no es muy consistente, porque uno espera que aparezca en todos los ejemplos. Pero hay en varios que no está, tiene el ejemplo pero no aparecen en la solución genérica)
(pidió que veamos el ejemplo en código)

PATRONES DE COMPORTAMIENTO

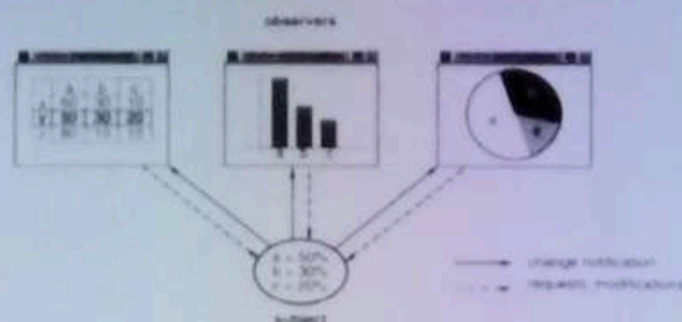
- Conciernen con algoritmos y la asignación de responsabilidades entre objetos.
- Los de clase utilizan herencia para distribuir comportamiento
- Los de objeto utilizan composición y describen como un grupo de objetos pares coopera para realizar una tarea.
- Pero disminuyendo las referencias explicitas entre sí para hacerlo débilmente acoplado

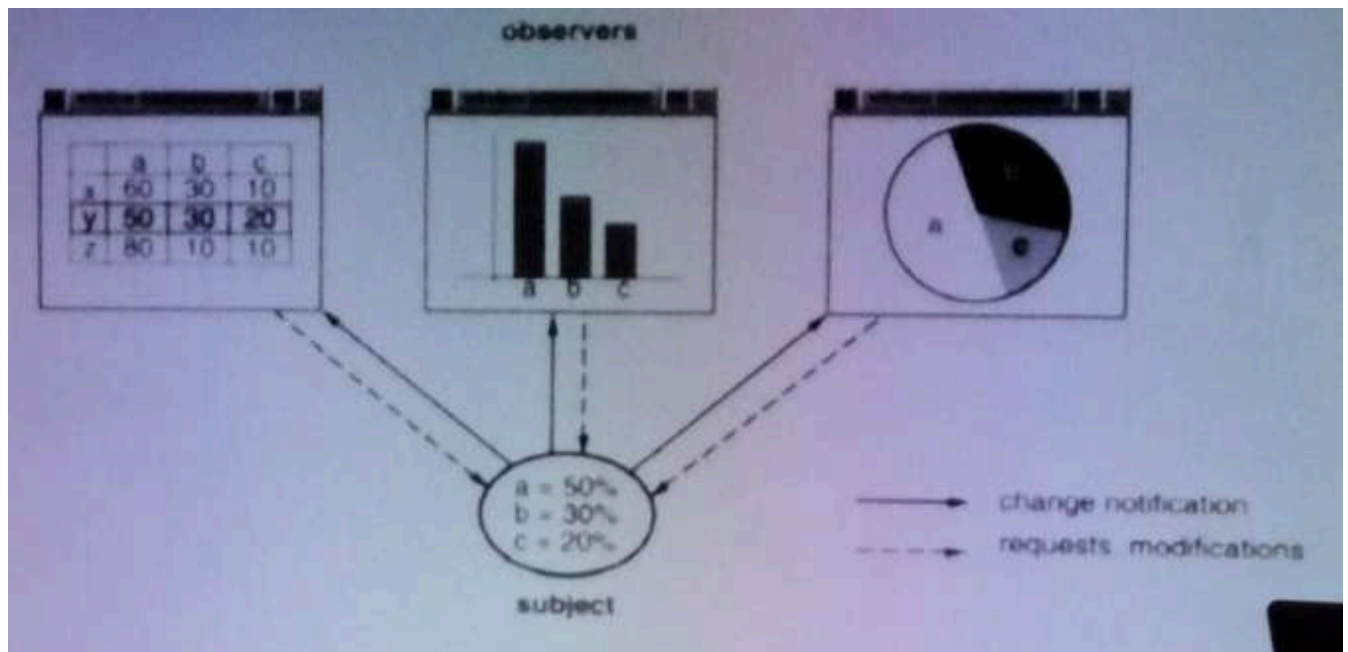
-acá es donde hay en mayor cantidad de patrones

OBSERVER

OBSERVER

- **Intención:** define una dependencia uno-a-muchos entre objetos de modo que cuando un objeto cambia el estado, todos sus dependientes son notificados y actualizados automáticamente.





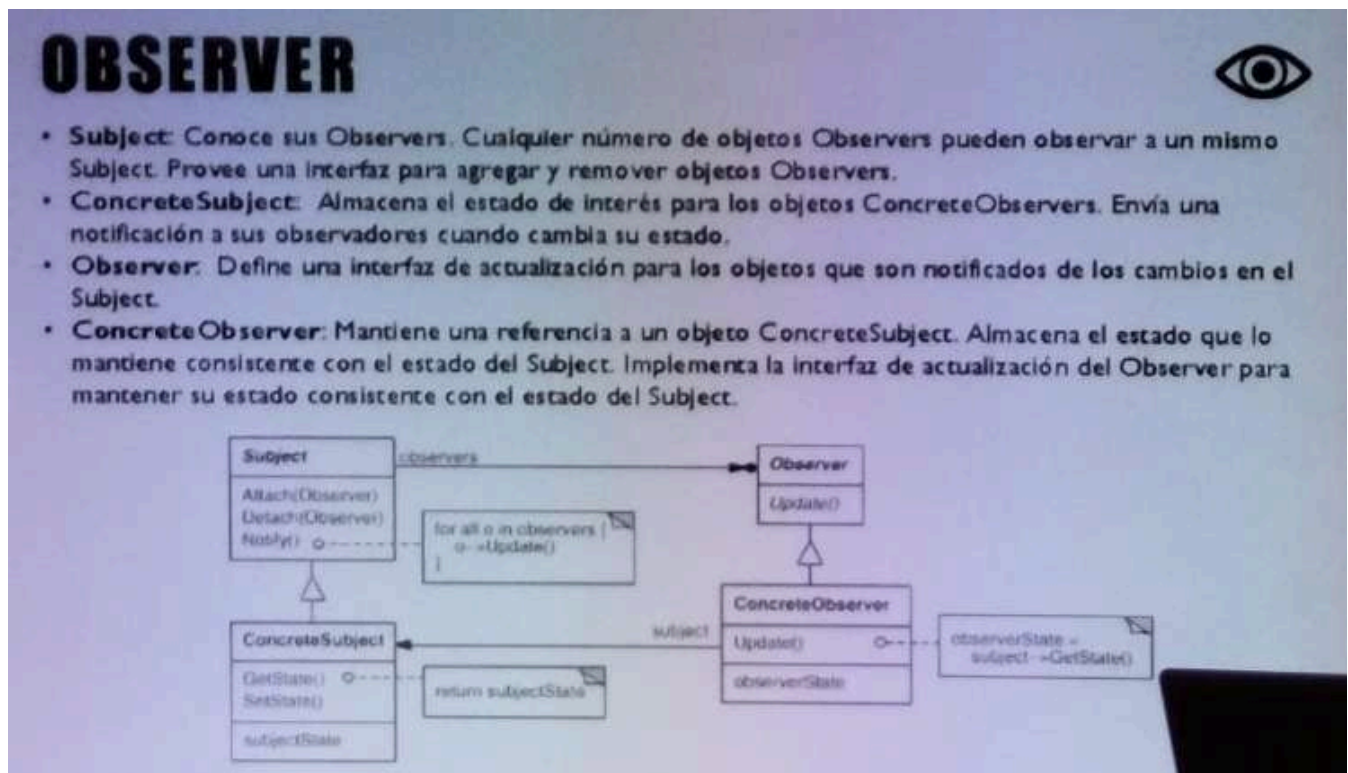
-este patrón es la base del patrón MVC

-representan una vista del mismo modelo, quieren mostrar ese contenido cuando sucede un cambio

-cuando sucede un cambio el Sujeto avisa a todos los Observadores. Los Observadores están suscritos al Sujeto para que pueda avisarles

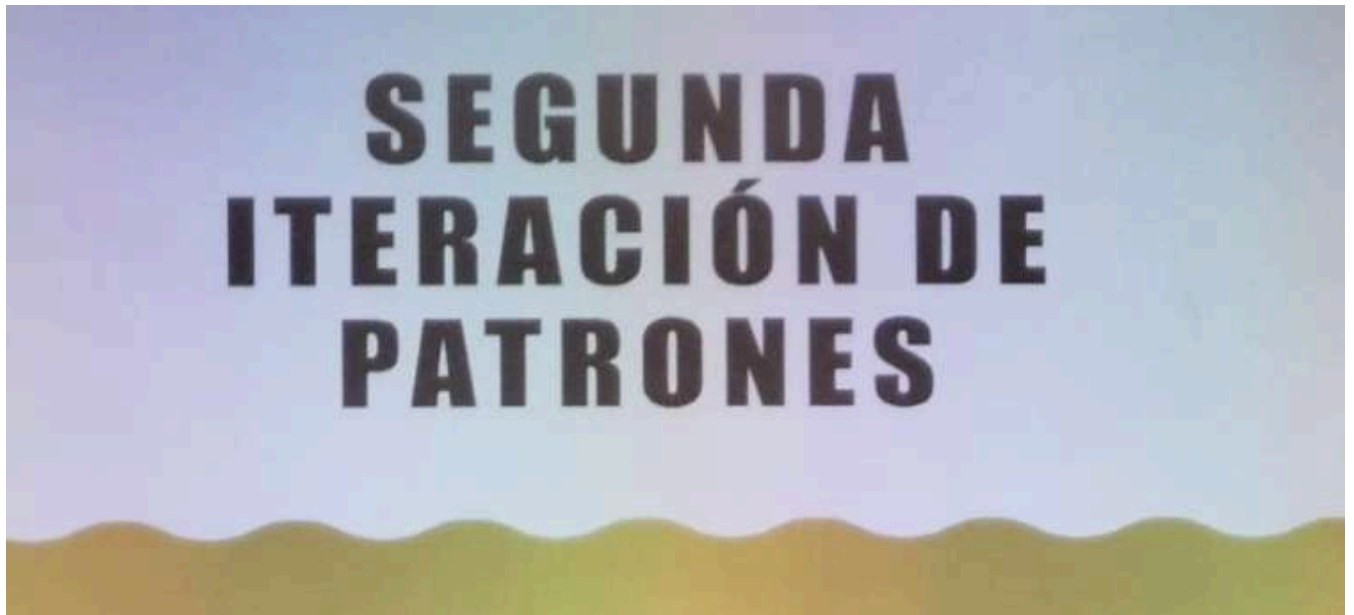
Aplicabilidad(power)

(otro power)



-El patrón también se llama Publicador-suscriptor

CLASE - 29/10



Esta clase hizo una dinámica de darnos situaciones y en 20 minutos que nosotros debamos indicar que patrón es y por qué. Con alejo y paula hicimos masomenos los diagramas de acuerdo al patrón en cada caso. Luego gaona explicó de forma breve cada patrón

Páginas con información sobre los patrones:

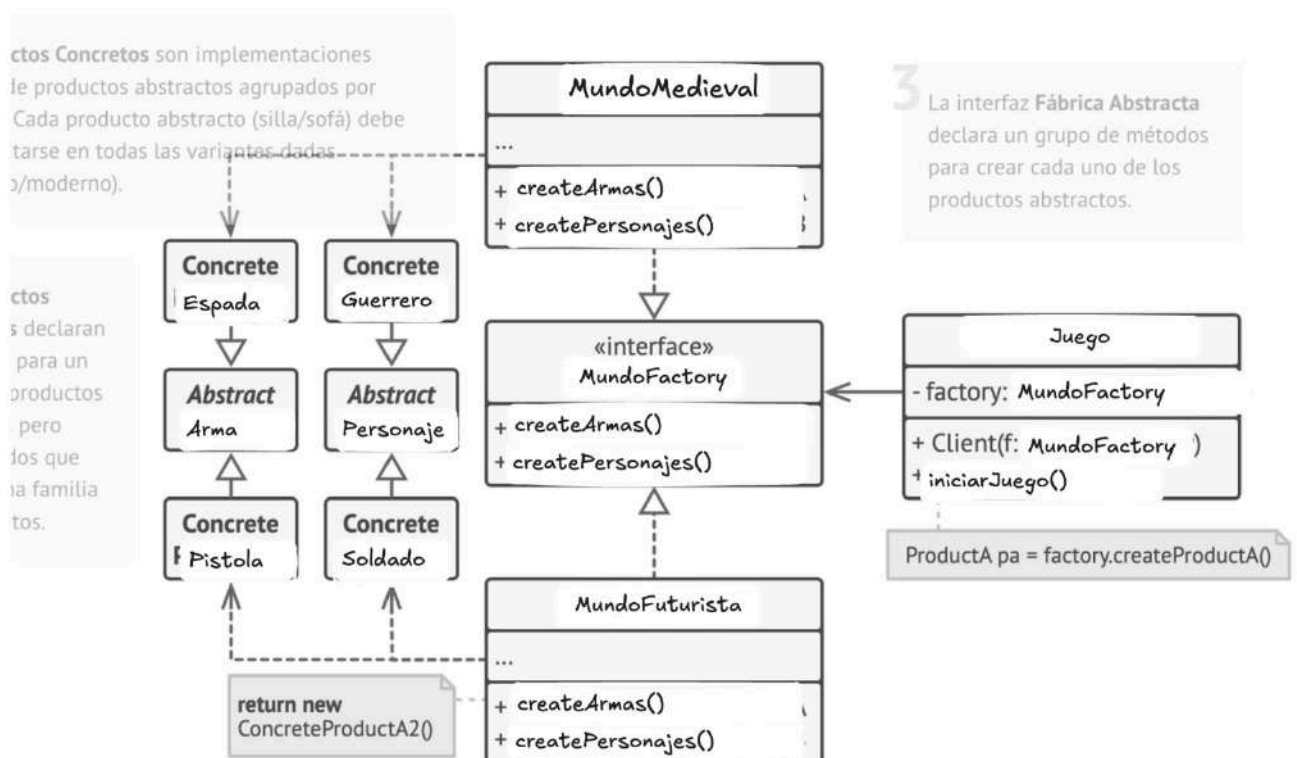
<https://refactoring.guru/design-patterns> (según lo que vi, esta página bastante recomendada)

<https://www.dofactory.com/net/design-patterns>

Ejemplo 1 - CREACIONAL

PATRÓN CREACIONAL: ?

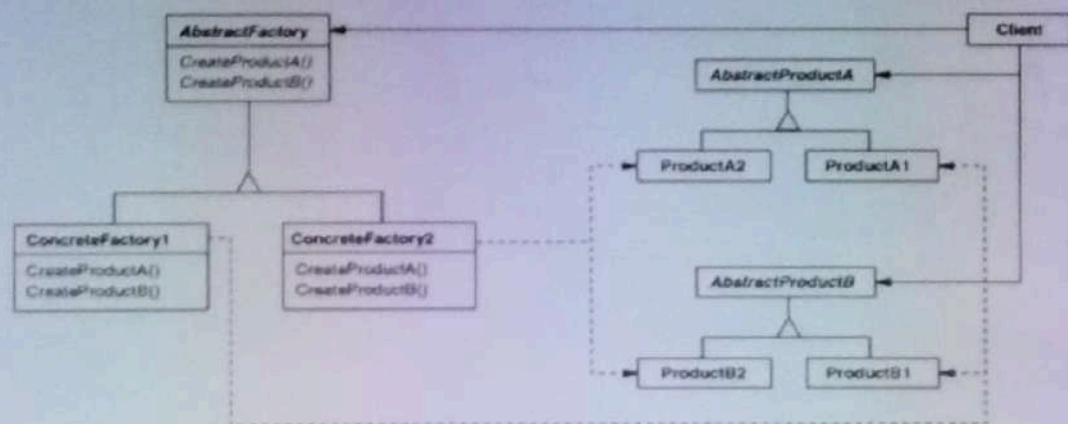
- Imagine que está desarrollando un videojuego con múltiples "mundos".
- Cada mundo tiene su propia estética y sus propios *personajes* y *armas*. Por ejemplo:
 - **Mundo Medieval:** Los personajes son guerreros, arqueros y magos, y las armas son espadas y arcos.
 - **Mundo Futurista:** Los personajes son soldados espaciales y robots, y las armas son pistolas láser y rifles de plasma.



ABSTRACT FACTORY

ABSTRACT FACTORY

- **Intención:** proveer una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas



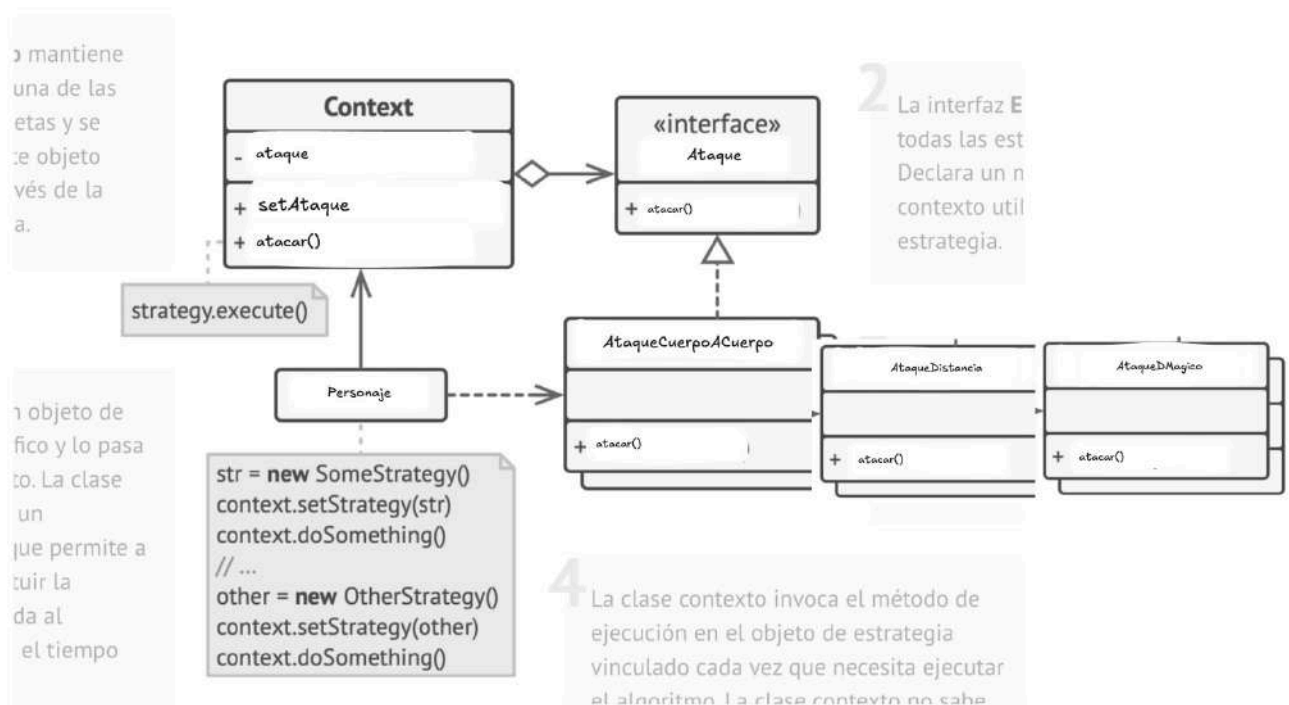
ABSTRACT FACTORY

- **Aplicabilidad:**
- un sistema debe ser **independiente** de cómo se crean, componen y representan sus productos.
- un sistema debe configurarse con **una de varias** familias de productos.
- **una familia** de objetos de productos relacionados **está diseñada para usarse en conjunto** y es necesario aplicar esta restricción.
- desea proporcionar una **biblioteca de clases de productos** y desea revelar solo sus interfaces, no sus implementaciones.

Ejemplo 2 - COMPORTAMIENTO

PATRÓN DE COMPORTAMIENTO: ?

- Continuando con el ejemplo. Cada personaje medieval puede usar diferentes estrategias de ataque:
 - Ataque cuerpo a cuerpo (p.ej. usado por guerreros con espadas).
 - Ataque a distancia (p.ej. usado por arqueros).
 - Ataque mágico (p.ej. usado por magos).
- Los personajes podrían cambiar su estrategia de ataque en tiempo de ejecución, dependiendo de la situación.

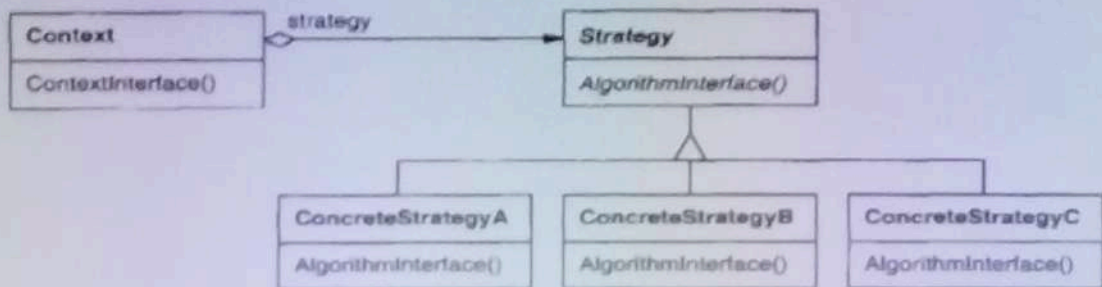


STRATEGY

STRATEGY



- **Intención:** definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables. La estrategia permite que el algoritmo varíe independientemente de los clientes que lo utilizan



STRATEGY

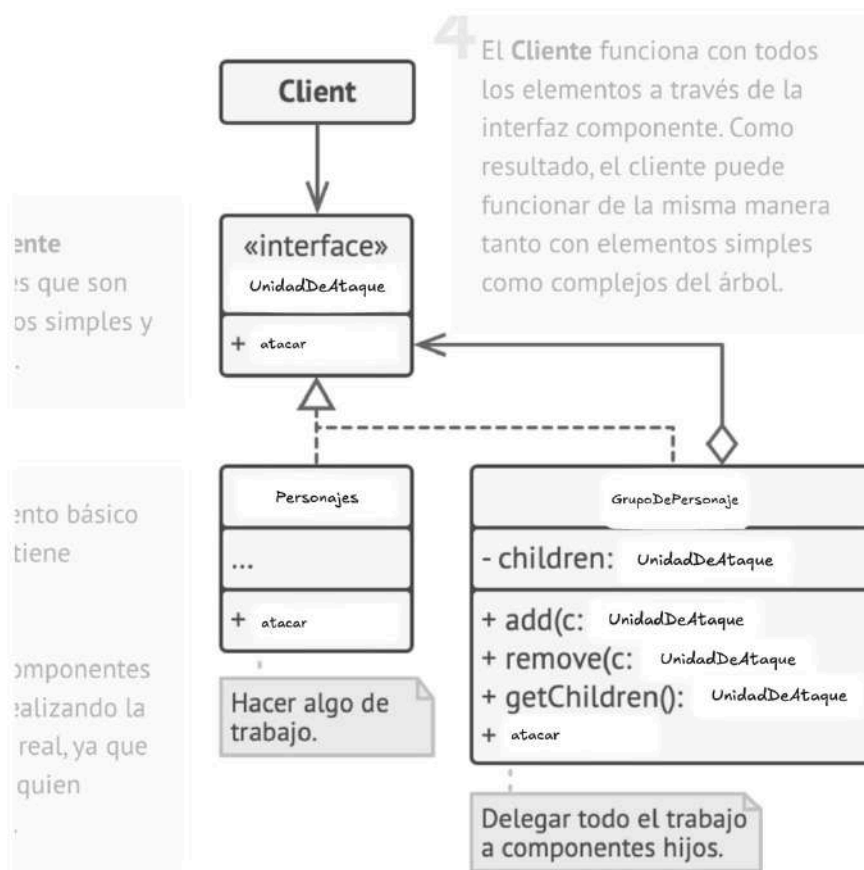


- **Aplicabilidad:**
 - muchas clases relacionadas difieren solo en su comportamiento.
 - necesita diferentes variantes de un algoritmo.
 - un algoritmo utiliza datos que los clientes no deberían conocer.
 - una clase define muchos comportamientos, y estos aparecen como múltiples declaraciones condicionales en sus operaciones.

Ejemplo 3 - ESTRUCTURAL

PATRÓN DE ESTRUCTURAL: ?

- Imaginemos que, en nuestro videojuego, los personajes pueden luchar de manera individual, pero **también es posible crear grupos (como ejércitos o equipos)** que pueden atacar o moverse como si fueran un solo personaje.
- Cada personaje tiene su propio ataque, pero los grupos pueden contener otros personajes y actuar de manera conjunta.
- El objetivo es tratar tanto a los personajes individuales como a los grupos de personajes de manera uniforme.

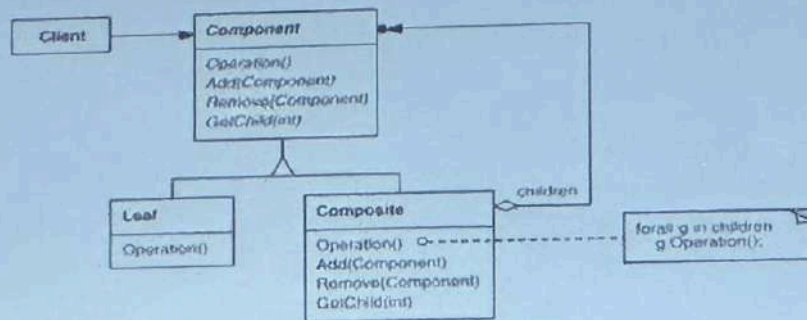


COMPOSITE

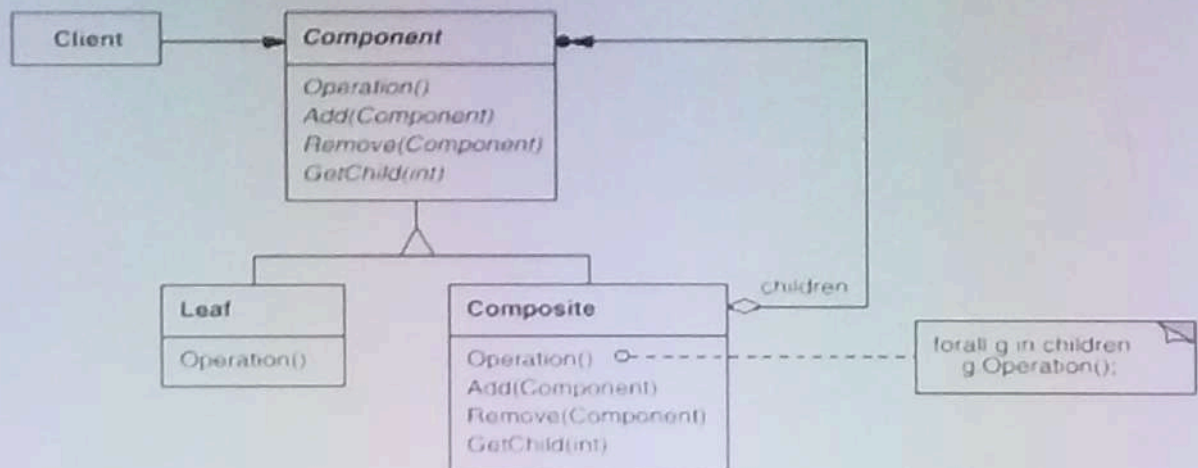


COMPOSITE

- **Intención:** compone objetos en estructuras arborescentes para representar jerarquías todo-partes. Composite permite a los clientes tratar objetos individuales y composiciones de objetos de manera uniforme.



a uniforme.



COMPOSITE



- Aplicabilidad:
- Desea representar **jerarquías todo-parte** de objetos.
- Desea que los clientes puedan ignorar la diferencia entre las composiciones de objetos y los objetos individuales. Los clientes **tratarán de manera uniforme** a todos los objetos de la estructura compuesta .

PRÁCTICA

Nos mando directo hacer la guía de Patrones. Dada una problemática indicar el patrón y hacer diagrama de clases o código

Dijo que en la práctica van a entrar los 9 que están en el resumen